

RAKTABEEJ

रक्तबीज · Dataset Generation Engine

Project Report

Developer	Arun VK
Repository	github.com/ArunVijaykumarclds/raktabeej
Live URL	https://raktabeej-5mbx.onrender.com/
Deployment	Render (Static Site)
License	MIT

1. Executive Summary

Raktabeej is a browser-based video-to-image-dataset generation engine built entirely with client-side web technologies. It allows users to upload a video file, configure how it should be segmented and sampled, and then extract hundreds or thousands of JPEG frames — all without sending any data to a server.

The project takes its name from the mythological demon Raktabeej from the Devi Mahatmya, whose power was the ability to multiply from every drop of his blood. The metaphor is applied deliberately: one source video becomes a structured army of labelled, traceable frame images — each inheriting its lineage from the parent video.

The core technical achievement is that the entire pipeline — video loading, duration detection, frame extraction, ZIP packaging, and manifest generation — runs inside the browser using only the native HTML5 Video element and Canvas API, with zero backend server dependency. Users can process sensitive video data locally with full confidence that nothing leaves their device.

Honest caveat: The project initially included FFmpeg WebAssembly as part of its architecture. That dependency remains in package.json and is bundled, but the actual extraction logic was replaced with a native browser implementation. The FFmpeg code is non-functional stubs kept only for API compatibility. This is clearly documented in the README.

2. Project Overview

2.1 Problem Statement

Creating image datasets from video footage is a common need in computer vision, machine learning, and research. The typical approaches involve:

- FFmpeg command-line tools — requiring technical knowledge and local installation
- Python scripts — requiring a development environment setup
- Cloud upload services — raising privacy and data security concerns
- Desktop software — platform-specific and often paid

None of these options are immediately accessible to researchers, students, or ML practitioners who want a fast, private, browser-based tool that works without any setup.

2.2 The Solution

Raktabeej provides a zero-install, zero-server, browser-native interface where users can:

- Drop a video file directly into the browser
- Configure segment duration and frames-per-segment with presets or custom values
- Watch frames extract in real time with live progress feedback

- Download individual frames, per-segment ZIPs, or a full labelled dataset archive
- Get a machine-readable JSON manifest encoding the full lineage of every frame

2.3 What Makes It Distinctive

- Privacy-first: no network request is made during processing — all data stays on device
- Zero dependencies in the critical path: frame extraction uses only native browser APIs
- Structured naming convention: every filename encodes its ancestry (video → segment → frame)
- Real-time gallery: frames appear as they are extracted, not after the job completes
- Fully deployed: accessible at a public URL, not just a local dev demo

3. Technical Architecture

3.1 Technology Stack

Layer	Technology	Version	Notes
Framework	React (StrictMode)	18.3.1	Core UI framework
Language	TypeScript	5.2.2	Full type safety across all services
Build Tool	Vite	5.3.1	Manual Rollup chunking for cache efficiency
Styling	Tailwind CSS	3.4.4	Custom Ancient Blood Chronicle theme
Extraction	HTML5 Video + Canvas	Browser native	Zero npm packages in the critical path
ZIP	JSZip	3.10.1	DEFLATE level 6 for all ZIP outputs
Icons	Lucide React	0.395.0	UI iconography
Fonts	Playfair Display, DM Sans, JetBrains Mono	Google Fonts	Serif, sans, monospace respectively
Deployment	Render (Static Site)	—	render.yaml zero-config deployment

3.2 Core Extraction Pipeline

The extraction pipeline is the heart of the project. It works entirely through browser-native APIs:

Step 1: Duration Detection

A temporary HTML video element is created with `preload='metadata'`. The video file is converted to an object URL and assigned as `src`. When the `onloadedmetadata` event fires, `video.duration` is read and the URL immediately revoked.

Step 2: Segment Building

The total duration is divided by the configured segment size using `Math.ceil`. Each segment receives a start time, an end time (clamped to the actual video duration to handle the last segment), and a status of 'pending'.

Step 3: Frame Extraction (Per Segment)

For each segment, a fresh hidden video element and canvas are created and appended to the document body. Frames are distributed evenly across the segment's time range:

- Single frame: lands at the segment midpoint (`progress = 0.5`)
- Multiple frames: evenly spaced from `startTime` to `endTime`
 - `progress = (f - 1) / (framesPerSegment - 1)`
 - `timestamp = startTime + progress × (endTime - startTime)`

For each timestamp: `video.currentTime` is set, the seeked event is awaited, the frame is painted to canvas using `ctx.drawImage` at native resolution, and `canvas.toBlob` encodes it as JPEG at 0.85 quality. The resulting Blob is wrapped in an object URL and emitted via callback to update the gallery in real time.

Step 4: Cleanup

After all frames in a segment complete, the object URL is revoked and the video/canvas elements are removed from the DOM. The loop then proceeds to the next segment. An `AbortController` signal is checked before every frame — allowing instant cancellation.

3.3 State Machine

The `useRaktabeej` hook manages the application state through clearly defined phases:

Phase	Description
idle	Initial state, awaiting file input
loading	Video file accepted, duration detection in progress
extracting	Sequential frame extraction loop active
done	All segments processed, dataset ready for export
error	An error occurred during loading or extraction
segmenting	Defined in types but never set at runtime (reserved)
zipping	Defined in types but never set at runtime (reserved)

Honest note: 'segmenting' and 'zipping' phases exist in the ProcessingPhase type and are checked by UI guards, but the hook never transitions into them at runtime. They are reserved for a future version.

3.4 Project File Structure

The codebase is cleanly organised into pages, components, hooks, services, utils, and types:

File / Module	Responsibility
pages/HomePage.tsx	Phase routing, gallery state, lightbox logic
hooks/useRaktabeej.ts	Central state machine, extraction orchestration
hooks/useDropZone.ts	Drag-and-drop events, type + size validation
hooks/useDownload.ts	Progress-tracked export wrappers
services/ffmpegService.ts	Duration detection + frame extraction (native browser)
services/downloadService.ts	Single frame, segment ZIP, full ZIP, manifest export
services/zipExtractService.ts	ZIP input handler for files over 2GB
utils/format.ts	Duration formatting, filename sanitization, segment math
types/index.ts	All shared TypeScript interfaces and enums

4. Features & Capabilities

4.1 Input

- Direct video upload up to 2 GB via drag-and-drop or file picker
- Supported formats: MP4, MOV, AVI, MKV, WEBM, MPEG/MPG, M4V
- ZIP archive input for files exceeding 2 GB (must contain exactly one video)
- Format badge display in the upload zone UI
- Two-ring pulse animation on drag-over for visual feedback

4.2 Configuration

Segment size presets: 500ms, 1s, 2s (default), 5s, 10s, plus custom (min 0.1s to full duration).

Frames per segment presets: 10, 25, 50 (default), 100, plus custom (1–500 integers).

A live extraction summary updates as the user adjusts settings, showing total segments and total expected frames before processing begins.

4.3 Processing

- Sequential extraction — one segment at a time, never parallel (avoids browser memory pressure)

- Real-time progress: dual progress bars (segments completed + frames within current segment)
- Dot-grid segment visualiser showing up to 60 segments with colour-coded status
- Live frame counter updating on every extracted frame
- Abort at any point — AbortController checked before every frame, resets to idle instantly
- Blood-drop SVG and 3-ring pulse animation during active extraction

4.4 Gallery & Export

- Grid view: 5 columns scaling to 10 at medium breakpoints, with skeleton shimmer for pending frames
- Timeline view: list layout with 64x36px thumbnail, timestamp in MM:SS.mmm, filename, hover actions
- Frame lightbox: full-screen modal (max 75vh), keyboard navigation with left/right/Escape, download
- Per-segment ZIP: named {videoBaseName}_segment_NN.zip with subfolder structure
- Full dataset ZIP: flat frames/ folder plus manifest.json, DEFLATE level 6
- Manifest-only download: raktabeej_manifest_{videoBaseName}.json
- Single frame download: synthetic anchor click, no server required

4.5 Output Format

Every frame is named using a strict convention that encodes its full lineage:

```
{sanitizedBaseName}_segment_{NN}_frame_{NNN}.jpg
```

Where NN is the 1-based, 2-digit zero-padded segment index and NNN is the 1-based, 3-digit zero-padded frame index. Zero-padding ensures correct sort order in file browsers, Python glob(), and ML dataset loaders.

The manifest.json file provides machine-readable lineage including video name, duration, total segments, frames per segment, total frames, segment size in seconds, generation timestamp, and a per-segment list of all frame filenames.

5. Design System

5.1 Visual Theme — Ancient Blood Chronicle

The project uses a custom Tailwind theme with a distinctive dark aesthetic that reflects its mythological name. The palette is deliberately matte — no gradients, no glow, no neon effects.

Token	Key Values	Purpose
blood	#c81e1e / #9b1c1c / #771d1d	Primary accent — buttons, progress bars, active states
charcoal	#1a1a1a / #0d0d0d / #080808	All background surfaces
copper	#cd7f32	Developer name highlight
parchment	Warm scale	Available in palette, sparsely used

5.2 Typography

Token	Font	Weights	Used for
font-serif	Playfair Display	400, 600, 700	Headings, logo, lore text
font-sans	DM Sans	300, 400, 500	Body, labels, buttons
font-mono	JetBrains Mono	400, 500	Timestamps, filenames, counters

5.3 Key Animations

- pulse-blood: Opacity and scale oscillation on the blood-drop icon during extraction (2s infinite)
- seed-expand: Scale 0.4 to 1.0 with opacity fade when a new frame card appears (0.4s)
- ring-expand: Expanding ring effect on the pulse circles (2s infinite)
- shimmer: Background-position sweep on skeleton placeholder cards (1.8s infinite)
- diffuse: Slow opacity animation on the background glow (8s infinite)

6. Deployment

6.1 Platform

The application is deployed as a static site on Render at <https://raktabeej-5mbx.onrender.com/>. The repository includes a render.yaml file that provides zero-config deployment — Render auto-detects it on push.

6.2 Build Configuration

Build command: `npm install && npm run build`. Output directory: `dist/`. The Vite build splits output into three browser-cached Rollup chunks:

- react: React and ReactDOM — changes infrequently, stays cached
- ffmpeg: `@ffmpeg/ffmpeg` and `@ffmpeg/util` — bundled but unused at runtime
- jszip: JSZip — only loaded when a user triggers export

6.3 HTTP Headers

The deployment sets Cross-Origin-Opener-Policy: same-origin and Cross-Origin-Embedder-Policy: require-corp headers on all routes. These were originally required for FFmpeg WebAssembly's SharedArrayBuffer support. Since FFmpeg is not actually used, these headers are now legacy configuration — they remain harmless but are no longer strictly necessary.

6.4 SPA Routing

A catch-all rewrite rule sends all paths to /index.html, enabling client-side routing. There are no cold starts since this is a static site with no server process.

Important limitation: Render's free tier spins down static sites after inactivity. The first load after a period of inactivity may be slow (10–30 seconds) while the service wakes up. This is a hosting constraint, not an application issue.

7. Use Cases & Target Audience

Domain	Application
Computer Vision	Training image classifiers and feature extractors from video
Object Detection	Frame sequences for bounding-box annotation tools (Roboflow, CVAT, LabelStudio)
Action Recognition	Dense frames from sports or gesture videos for VideoMAE, SlowFast models
Medical Imaging	Frames from endoscopy or surgical video — fully offline, nothing leaves the device
Remote Sensing	Aerial and satellite video frame extraction
Content Moderation	Frame sampling from uploads into a classifier pipeline, no server required
Optical Flow	Consecutive frames from the same segment for flow estimation training
Research & Academia	Reproducible dataset generation with machine-readable manifest for publications

8. Honest Assessment

8.1 What Works Well

- The core extraction pipeline is solid and genuinely useful for its stated purpose

- Privacy guarantee is real — no network traffic during processing, verifiable in DevTools
- The naming convention and manifest design are thoughtful and production-ready for ML pipelines
- The UI is polished and the design system is cohesive and distinctive
- Real-time gallery feedback makes the tool feel responsive and satisfying to use
- The abort mechanism works correctly — AbortController is properly threaded through the stack
- The codebase is well-structured with clean separation of concerns across hooks, services, and utils
- TypeScript is used consistently and meaningfully, not just as a formality

8.2 Known Limitations & Honest Gaps

- **FFmpeg is listed in package.json and bundled, but is non-functional. The README is transparent about this, but the build still carries dead weight that inflates the bundle size.**
 - The ffmpeg chunk is always downloaded by the browser even though no code calls it at runtime.
- **The revokeFrameUrls function in downloadService.ts is exported but never called anywhere in the application.**
 - This means extracted frame Blob URLs accumulate in memory for the lifetime of the browser session.
 - For large extractions (thousands of frames), this could cause memory pressure.
- **'segmenting' and 'zipping' phases are defined in the type system and checked by UI guards but never set at runtime.**
 - The UI guards using isProcessing and isActive work correctly, but the phase transitions they guard are never triggered.
- **video/MP2T (.ts files) is included in SUPPORTED_VIDEO_TYPES and passes isVideoFile() validation, but is not in the UI badge list, not in the file input accept attribute, and not in the extension list.**
 - A .ts file with the correct MIME type would pass validation but the user is never told this is supported.
- **GitHub cannot stream or preview the MP4 demo files in the repository — they are stored but not playable.**
 - For a project that demonstrates video processing, this is a notable gap in the repository presentation.
 - Resolution: host demo videos on Cloudinary or YouTube and embed or link them from the README.
- **The COOP/COEP headers add a constraint that might break some third-party embeds or cross-origin resources, even though the original reason for them (FFmpeg SharedArrayBuffer) no longer applies.**

- **No automated tests are present in the repository.**
 - The extraction logic, segment math, filename sanitization, and ZIP generation have no test coverage.
 - format.ts utility functions like computeSegmentCount, sanitizeFilename, and generateFrameFilename are strong candidates for unit testing.
- **Performance on very long videos or high frame counts is untested in the documentation.**
 - The sequential extraction model avoids parallel memory pressure but may be slow for videos requiring 10,000+ frames.

8.3 What Could Be Improved

- Remove or lazy-load the FFmpeg bundle to reduce initial page weight
- Call revokeFrameUrls on reset to free Blob URL memory
- Add unit tests for format.ts and segment math utilities
- Host demo videos externally and embed them in the README for repository visitors
- Remove the COOP/COEP headers or document clearly why they remain
- Fill in the screenshot placeholders in the README with actual UI captures

9. Future Roadmap (As Stated by Developer)

The README documents a planned evolution called Raktabeej AI. The following is the developer's stated roadmap — none of these capabilities exist in the current release:

Stage	Status	Description
Frame Extraction	Complete	HTML5 Video + Canvas, JPEG 0.85, named frames
Dataset Manifest	Complete	Machine-readable JSON lineage for every frame
AI Captioning	Future	Per-frame scene descriptions from vision-language models
Object Detection	Future	Bounding box inference over extracted frames
Auto Annotation	Future	Structured label assignment from detection outputs
Training-Ready Export	Future	Frames + captions + annotations + manifest in one archive

10. Conclusion

Raktabeej is a well-executed, genuinely useful tool that solves a real problem in a distinctive way. The decision to build entirely on browser-native APIs rather than FFmpeg WebAssembly — while keeping the FFmpeg infrastructure stubbed out — results in a leaner, more reliable extraction pipeline that works consistently across browsers without SharedArrayBuffer concerns.

The design system is cohesive, the code architecture is clean, and the output format is thoughtfully designed for downstream ML workflows. The manifest structure, zero-padded naming convention, and real-time gallery feedback all reflect considered engineering decisions rather than quick implementation choices.

The honest gaps — the dead FFmpeg bundle, the unreleased revokeFrameUrls, the phantom phase transitions, and the absence of tests — are real but not critical. They represent the difference between a working project and a production-hardened one. The README itself is unusually transparent about these limitations, which is a mark of developer maturity.

For a browser-only, zero-backend, privacy-first dataset generation tool, Raktabeej does what it says. The core pipeline works. The deployment is live. The output is real and usable by ML practitioners today.

One seed. Countless descendants.